UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

JEAMPIERRE ORTIZ

PABLO TOAPANTA

ESTIVEN CHILUISA

DAVID RODRIGUEZ

Ing. JOSÉ CAIZA, Mg

TERCERO SOFTWARE - “B”

Ambato - Ecuador

1. Definición del Problema

Un laboratorio académico necesita un sistema informático para gestionar la atención de estudiantes. por lo que el sistema necesita cumplir con los siguientes parámetros.

- Atender a los estudiantes estrictamente en el orden en que llegan, garantizando equidad.
- Mantener un registro de todas las solicitudes que ya han sido atendidas a lo largo del día.
- Proveer un mecanismo de consulta rápida que permita visualizar el último registro procesado de forma inmediata, utilizando una estructura de acceso tipo LIFO.

2. Justificación de las Estructuras de Datos Elegidas

El problema nos expone comportamientos que coinciden con las tres Estructuras de Datos Abstractas (TDA). Se opta por separar las responsabilidades para cada estructura:

- **Cola (Queue) para Solicitudes Pendientes:** Se justifica porque modela matemáticamente una fila de espera en la vida real. Opera bajo la política **FIFO** (First In, First Out). Esto garantiza que el primer estudiante que encola su solicitud sea obligatoriamente el primero en desencolar para ser atendido.
- **Lista Enlazada (List) para Historial Completo:** Se justifica por su capacidad de crecimiento dinámico. Como sabemos, una lista no tiene un límite máximo predefinido, lo que permite almacenar un historial infinito de todas las transacciones del día, insertando siempre al final para mantener el orden cronológico.
- **Pila (Stack) para Consultas Recientes:** Se justifica por su política **LIFO** (Last In, First Out). En un historial largo, buscar los últimos registros requiere recorrer muchos datos. La pila soluciona esto obligando a que el último estudiante atendido se posicione en el tope, permitiendo que la consulta de "recientes" sea instantánea.

3. Casos de Prueba (Comprobación de Lógica)

Para validar que el sistema funciona bajo las reglas establecidas, se definen los siguientes 3 casos de prueba críticos:

- **Caso 1: Intentar atender con la Cola Vacía (Prevención de fallos)**
    - *Acción:* El sistema intenta ejecutar la función de "atender siguiente" cuando no hay estudiantes registrados.
    - *Resultado comprobado:* El sistema detecta la ausencia de nodos (frente nulo), aborta la extracción de datos y emite una advertencia, evitando que el programa colapse.

Código

Método

```
// Atiende al primero de la cola y lo mueve al historial y a recientes
void atenderSiguiente() {
    if (pendientes.estaVacia()) {
        cout << "[!] No hay estudiantes esperando en la cola.\n";
        return;
    }

    Solicitud atendida = pendientes.desencolar();

    // Guardamos en ambas estructuras según lo requerido
    historial.agregar(atendida);
    recientes.apilar(atendida);

    cout << "[v] Solicitud ATENDIDA: " << atendida.nombreEstudiante << "\n";
}
```

Invocación al método

```
// ==========================================
int main() {
    Laboratorio lab;

    cout << "==========================================\n";
    cout << "       EJECUCION DE CASOS DE PRUEBA      \n";
    cout << "==========================================\n";

    // -------------------------------------------------------
    // PRUEBA 1: Cola vacía
    // -------------------------------------------------------
    cout << "\n[PRUEBA 1] Cola vacia: intentar atender sin registros\n";
    // Comportamiento esperado: El sistema debe detectar que está vacía y no fallar.
    lab.atenderSiguiente();
```

Ejecución

```
[PRUEBA 1] Cola vacia: intentar atender sin registros
[!] No hay estudiantes esperando en la cola.

[PRUEBA 2] Un estudiante: agregar y atender una sola vez
[+] Solicitud recibida: Zoe - Uso de multimetro
[v] Solicitud ATENDIDA: Zoe

--- COLA DE ESPERA (Pendientes) ---
  - No hay solicitudes pendientes.
```

- **Caso 2: Verificación de la política FIFO en Pendientes**
    - *Acción:* Ingresan consecutivamente los estudiantes "Alberto", "Beatriz" y "Carmen". Luego, se atiende una sola solicitud.
    - *Resultado comprobado:* El sistema extrae a "Alberto" (el más antiguo) y lo transfiere. La cola de espera retiene únicamente a "Beatriz" y "Carmen", en ese orden exacto.

Código

```
    // ---------------------------------------------------------
// CASO 2: Verificación de la política FIFO en Pendientes
// ---------------------------------------------------------
cout << "=== CASO 2: Verificacion de la politica FIFO en Pendientes ===" << endl;

// Acción: Ingresan consecutivamente los estudiantes
lab.recibirSolicitud("Alberto", "Uso de Cautin");
lab.recibirSolicitud("Beatriz", "Fuente de poder");
lab.recibirSolicitud("Carmen", "Protoboard");

cout << "\nAccion: Se atiende una sola solicitud." << endl;
// El sistema debe extraer a "Alberto" (el más antiguo)
lab.atenderSiguiente();

// Resultado comprobado: La cola retiene únicamente a "Beatriz" y "Carmen"
cout << "\nResultado comprobado - Estado de la cola de espera:";
lab.verPendientes();

cout << "\n-------------------------------------------------------------\n\n";
```

```
public:
    // Recibe una solicitud y la pone en la cola (espera)
    void recibirSolicitud(string nombre, string descripcion) {
        Solicitud nueva = {nombre, descripcion};
        pendientes.encolar(nueva);
        cout << "[+] Solicitud recibida: " << nombre << " - " << descripcion << "\n";
    }

    // Atiende al primero de la cola y lo mueve al historial y a recientes
    void atenderSiguiente() {
        if (pendientes.estaVacia()) {
            cout << "[!] No hay estudiantes esperando en la cola.\n";
            return;
        }

        Solicitud atendida = pendientes.desencolar();

        // Guardamos en ambas estructuras según lo requerido
        historial.agregar(atendida);
        recientes.apilar(atendida);

        cout << "[v] Solicitud ATENDIDA: " << atendida.nombreEstudiante << "\n";
    }
```

Ejecución

```
=========================================
=== CASO 2: Verificacion de la politica FIFO en Pendientes ===
[+] Solicitud recibida: Alberto - Uso de Cautin
[+] Solicitud recibida: Beatriz - Fuente de poder
[+] Solicitud recibida: Carmen - Protoboard

Accion: Se atiende una sola solicitud.
[v] Solicitud ATENDIDA: Alberto

Resultado comprobado - Estado de la cola de espera:
--- COLA DE ESPERA (Pendientes) ---
  - [Beatriz] Fuente de poder
  - [Carmen] Protoboard

-------------------------------------------------------------
```

- **Caso 3: Verificación de la política LIFO en Recientes**
  - *Acción:* Tras atender a "Alberto", "Beatriz" y "Carmen" (en ese orden), se solicita ver los últimos atendidos.
  - *Resultado comprobado:* La pila muestra a "Carmen" en la primera posición (cima), seguida de "Beatriz" y finalmente "Alberto", demostrando que la inversión del orden cronológico funciona para la consulta rápida.

Código

```
    void verUltimosAtendidos() {
        cout << "\n--- ULTIMOS ATENDIDOS (Recientes) ---\n";
        recientes.mostrarRecientes(3); // Muestra los últimos 3 por defecto
    }
};
```

```
// -------------------------------------------------------
// CASO 3: Verificación de la política LIFO en Recientes
// -------------------------------------------------------
cout << "=== CASO 3: Verificacion de la politica LIFO en Recientes ===" << endl;

// Acción: Tras atender a Alberto (en el caso anterior), se atiende a Beatriz y Carmen.
cout << "Accion: Se atiende a Beatriz y Carmen en ese orden cronologico." << endl;
lab.atenderSiguiente(); // Atiende a Beatriz
lab.atenderSiguiente(); // Atiende a Carmen

// Resultado comprobado: La pila muestra a Carmen en la cima, seguida de Beatriz y Alberto.
cout << "\nResultado comprobado - Ultimos atendidos (Inversion cronologica):";
lab.verUltimosAtendidos();

return 0;
}
```

Ejecución

```
=== CASO 3: Verificacion de la politica LIFO en Recientes ===
Accion: Se atiende a Beatriz y Carmen en ese orden cronologico.
[v] Solicitud ATENDIDA: Beatriz
[v] Solicitud ATENDIDA: Carmen

Resultado comprobado - Ultimos atendidos (Inversion cronologica):
--- ULTIMOS ATENDIDOS (Recientes) ---
  - [Carmen] Protoboard
  - [Beatriz] Fuente de poder
  - [Alberto] Uso de Cautin
```

4. Conclusión

La lógica del sistema de laboratorio se soluciona de manera natural y óptima mediante la utilización conjunta de una lista, una cola y una pila. Cada estructura seleccionada utiliza en vez de una única estructura que se obliga a través de complejos algoritmos de búsqueda o ordenamiento (que consumiría un tiempo innecesario de procesamiento), sus propias reglas de acceso innatas (FIFO y LIFO) para estructurar la información automáticamente cuando se inserta y extrae. Esto produce un código modular, seguro y de alta eficiencia para gestionar recursos.

ANEXOS

EJECUCIÓN

```
========================================
       EJECUCION DE CASOS DE PRUEBA
========================================

[PRUEBA 1] Cola vacia: intentar atender sin registros
[!] No hay estudiantes esperando en la cola.

[PRUEBA 2] Un estudiante: agregar y atender una sola vez
[+] Solicitud recibida: Zoe - Uso de multimetro
[v] Solicitud ATENDIDA: Zoe

--- COLA DE ESPERA (Pendientes) ---
  - No hay solicitudes pendientes.

[PRUEBA 3] Varios estudiantes: verificar FIFO en pendientes
[+] Solicitud recibida: Alberto - Cautin
[+] Solicitud recibida: Beatriz - Fuente de poder
[+] Solicitud recibida: Carmen - Protoboard

Estado inicial FIFO (Debe mostrar: Alberto -> Beatriz -> Carmen):

--- COLA DE ESPERA (Pendientes) ---
  - [Alberto] Cautin
  - [Beatriz] Fuente de poder
  - [Carmen] Protoboard

Atendiendo al primero (Debe salir Alberto):
[v] Solicitud ATENDIDA: Alberto

Estado despues de atender (Debe quedar: Beatriz -> Carmen):

--- COLA DE ESPERA (Pendientes) ---
```

```
Atendiendo al primero (Debe salir Alberto):
[v] Solicitud ATENDIDA: Alberto

Estado despues de atender (Debe quedar: Beatriz -> Carmen):

--- COLA DE ESPERA (Pendientes) ---
  - [Beatriz] Fuente de poder
  - [Carmen] Protoboard

[PRUEBA 4] Historial: revisar orden completo cronologico
[v] Solicitud ATENDIDA: Beatriz
[v] Solicitud ATENDIDA: Carmen

--- HISTORIAL COMPLETO (Atendidos) ---
  - [Zoe] Uso de multimetro
  - [Alberto] Cautin
  - [Beatriz] Fuente de poder
  - [Carmen] Protoboard

[PRUEBA 5] Recientes: confirmar ultimo atendido arriba

--- ULTIMOS ATENDIDOS (Recientes) ---
  - [Carmen] Protoboard
  - [Beatriz] Fuente de poder
  - [Alberto] Cautin
```